



TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

**XÂY DỰNG SELF-EVOLVING AI AGENTS
TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG**

Nguyễn Thanh Hải

optimus1072005@gmail.com

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Data Science & AI Engineering

Mentor: Nguyễn Duy Hưng

Đơn vị: Tổng công ty Mạng lưới Viettel

HÀ NỘI - 2026

Tóm tắt

Đề tài nghiên cứu hệ thống **Self-Evolving AI Agents** hỗ trợ xử lý sự cố mạng viễn thông trên nền tảng **NIKA**. Trong môi trường mạng hiện đại, sự cố có thể xuất hiện từ nhiều nguồn khác nhau như sai cấu hình, lỗi định tuyến, lỗi dịch vụ, tấn công mạng hoặc bất thường ở tầng dữ liệu. Các agent dựa trên LLM có khả năng suy luận và sử dụng công cụ, nhưng nếu chỉ vận hành với prompt và mô tả công cụ cố định, chúng khó thích nghi với các dạng lỗi mới, dễ lặp lại sai lầm và chưa tận dụng được kinh nghiệm từ các lần chẩn đoán trước. Vì vậy, hướng tiếp cận evolving agents được lựa chọn nhằm giúp hệ thống tự cải thiện dần trong quá trình xử lý sự cố.

Hệ thống được xây dựng bằng cách kết hợp agent suy luận dựa trên LLM, môi trường mô phỏng Kathará và các công cụ chẩn đoán qua MCP. Agent thực hiện ba nhiệm vụ chính: phát hiện sự cố, định vị thiết bị lỗi và phân tích nguyên nhân gốc rễ. Đóng góp của đề tài tập trung vào hai mô-đun tự tiến hóa. Thứ nhất, **Procedural Memory**, lấy cảm hứng từ hướng procedural memory trong Skill-Pro [3], lưu trữ các kỹ năng chẩn đoán theo dạng quy trình để agent có thể truy xuất và tái sử dụng kinh nghiệm phù hợp với ngữ cảnh mạng hiện tại. Thứ hai, **Tool Refinement**, lấy cảm hứng từ DRAFT [7], tinh chỉnh tài liệu mô tả công cụ MCP dựa trên các lần gọi công cụ thực tế, giúp agent hiểu rõ hơn điều kiện sử dụng, tham số và cách diễn giải kết quả trả về.

Thực nghiệm trên benchmark NIKA cho thấy việc bổ sung cơ chế tự tiến hóa giúp hệ thống cải thiện chất lượng chẩn đoán so với agent thông thường. Agent có xu hướng sử dụng công cụ hợp lý hơn, giảm các bước điều tra không cần thiết và tích lũy được kinh nghiệm từ những incident đã xử lý. Kết quả này cho thấy evolving agents là một hướng tiếp cận phù hợp cho bài toán xử lý sự cố mạng, nơi môi trường thay đổi liên tục và kinh nghiệm vận hành có vai trò quan trọng đối với chất lượng chẩn đoán.

Từ khóa: *Self-Evolving AI Agents, xử lý sự cố mạng, NIKA, bộ nhớ quy trình tiến hóa, tiến hóa công cụ, Kathará.*

Lời cảm ơn

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới người đã hướng dẫn và đồng hành cùng em trong dự án, anh **Nguyễn Duy Hưng**, chuyên viên Khoa học Dữ liệu, Trung tâm Chuyển đổi số, Tổng công ty Mạng lưới Viettel, anh **Bùi Quang Hưng** ... Trong suốt thời gian thực hiện dự án, sự hỗ trợ và động viên của anh không chỉ thúc đẩy và giúp đỡ em về mặt chuyên môn mà còn định hướng cho sự phát triển của em trên con đường theo đuổi sự nghiệp sau này.

Đồng thời, em cũng xin gửi lời cảm ơn tới tất cả giảng viên đào tạo lĩnh vực **Data Science & AI Engineer** tại chương trình **Viettel Digital Talent 2026**. Kiến thức và những chia sẻ từ các anh chị đã và sẽ giúp đỡ chúng em rất nhiều trong quá trình học tập, làm việc và phát triển trong tương lai. Bên cạnh đó, em cũng xin gửi lời cảm ơn tới tất cả các bạn thực tập sinh trong lĩnh vực Data Science & AI Engineer nói riêng và toàn chương trình nói chung vì đã giúp đỡ và đồng hành cùng nhau trong quá trình học tập tại chương trình năm nay.

Cuối cùng, em xin cảm ơn **Tập đoàn Công nghiệp - Viễn thông Quân đội Viettel** vì đã cho em cơ hội tham gia và học tập tại chương trình VDT 2026, một hành trình với không chỉ kiến thức mà còn là những kinh nghiệm và cơ hội mới, là hành trang cho hành trình theo đuổi tương lai.

Lời cam đoan

Tôi, Nguyễn Thanh Hải, sinh viên trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, thực tập sinh lĩnh vực đào tạo Data Science & AI Engineer, chương trình Viettel Digital Talent 2026, xin cam đoan dự án với đề tài “Xây dựng Self-Evolving AI Agents trong xử lý sự cố mạng viễn thông” là sản phẩm nghiên cứu gốc của tôi. Tôi xin cam đoan toàn bộ nội dung trình bày trong báo cáo hoàn toàn không sao chép từ bất kỳ tài liệu nào khác. Ngoài ra, tôi cũng không sử dụng bất kỳ một phát biểu, kết quả hay công trình nghiên cứu của người khác mà không trích dẫn đầy đủ. Tôi xin cam mã nguồn của dự án này là do tôi tự phát triển, không sao chép mã nguồn của người khác. Tôi xin chịu hoàn toàn trách nhiệm theo quy định đối với các cam kết của mình.

Thực tập sinh

Nguyễn Thanh Hải

Mục lục

Tóm tắt	iii
Lời cảm ơn	iv
Lời cam đoan	v
Mục lục	vi
Danh mục hình ảnh	vii
1 Giới thiệu chung	1
1.1 Đặt vấn đề	1
1.2 Phát biểu bài toán	2
1.3 Đóng góp của dự án	3
2 Kiến thức nền tảng	5
2.1 Xử lý sự cố mạng và phân tích nguyên nhân gốc rễ	5
2.2 Mô hình ngôn ngữ lớn và AI Agent	6
2.3 Self-Evolving AI Agents	6
2.4 Các nghiên cứu liên quan trực tiếp	7
3 Kiến trúc hệ thống	8
3.1 Kiến trúc tổng quan	8
3.2 Workflow chẩn đoán	9
3.3 Procedural Memory	9
3.4 Tool Refinement	10
4 Thực nghiệm	11
4.1 Thiết lập thực nghiệm	11
4.2 Benchmark và chỉ số đánh giá	12
4.3 Kết quả thực nghiệm	12
4.3.1 Các bảng và hình bổ sung dự kiến	13

4.4	Phân tích hiệu quả và hạn chế	15
5	Kết luận và hướng phát triển	16
5.1	Kết quả đạt được của dự án	16
5.2	Những hạn chế hiện tại	17
5.3	Định hướng phát triển tương lai	17
	Tài liệu tham khảo	19

Danh sách hình vẽ

4.1	Placeholder figure: xu hướng RCA và Localization theo số case tăng dần. Số liệu minh họa, chưa dùng làm kết quả chính thức.	14
4.2	Placeholder figure: so sánh độ ổn định giữa baseline và evolved qua 3 lần chạy. Số liệu minh họa, chưa dùng làm kết quả chính thức.	14
4.3	Placeholder figure: so sánh ReAct trên GPT-OSS-120B và Qwen3.5- 122B-A10B-FP8. Số liệu minh họa, chưa dùng làm kết quả chính thức. .	14

Chương 1

Giới thiệu chung

Chương này trình bày góc nhìn tổng quan về đề tài nghiên cứu, bao gồm ba nội dung chính. Đầu tiên, phần 1.1 đưa ra bối cảnh, động lực và lý do lựa chọn đề tài. Tiếp theo, phần 1.2 phát biểu cụ thể bài toán cần giải quyết trong dự án. Cuối cùng, phần 1.3 trình bày những đóng góp chính của dự án trong việc nghiên cứu và xây dựng hệ thống Self-Evolving AI Agents hỗ trợ xử lý sự cố mạng viễn thông.

1.1 Đặt vấn đề

Sự phát triển nhanh chóng của các hệ thống mạng viễn thông hiện đại đã đặt ra nhiều thách thức lớn trong quá trình vận hành, giám sát và xử lý sự cố. Các mạng thế hệ mới, đặc biệt là mạng di động 5G, có quy mô ngày càng lớn, cấu trúc ngày càng phức tạp và bao gồm nhiều thành phần có quan hệ phụ thuộc chặt chẽ với nhau như trạm gốc, thiết bị vô tuyến, lõi mạng, cấu hình truyền dẫn, tham số handover và các chỉ số hiệu năng mạng. Trong môi trường đó, sự cố có thể xuất hiện từ nhiều nguyên nhân khác nhau, từ lỗi phần cứng, sai cấu hình, nhiễu tín hiệu, suy giảm vùng phủ cho đến quá tải tài nguyên. Nếu không được phát hiện và xử lý kịp thời, các sự cố này có thể làm suy giảm chất lượng dịch vụ, gây gián đoạn kết nối, ảnh hưởng trực tiếp đến trải nghiệm người dùng và làm tăng chi phí vận hành của nhà mạng.

Trong thực tế, xử lý sự cố mạng viễn thông không chỉ dừng lại ở việc phát hiện dấu hiệu bất thường, mà còn yêu cầu xác định nguyên nhân gốc rễ đứng sau các hiện tượng quan sát được. Đây là bài toán khó vì dữ liệu vận hành mạng thường có kích thước lớn, nhiều chiều, thay đổi liên tục theo thời gian và chứa các mối quan hệ nhân quả phức tạp giữa nhiều thành phần. Các phương pháp truyền thống thường dựa vào luật thủ công, kinh nghiệm của chuyên gia hoặc các mô hình học máy tĩnh. Tuy nhiên, những phương

pháp này gặp hạn chế khi hệ thống mạng thay đổi, khi xuất hiện các dạng lỗi mới hoặc khi cần đưa ra lời giải thích rõ ràng cho quá trình chẩn đoán.

Trong những năm gần đây, các mô hình ngôn ngữ lớn và AI Agent mở ra nhiều tiềm năng mới cho bài toán xử lý sự cố nhờ khả năng phân tích dữ liệu, suy luận theo ngữ cảnh, sử dụng công cụ và sinh giải thích bằng ngôn ngữ tự nhiên. Tuy vậy, các agent thông thường vẫn mang tính tĩnh, tức là hoạt động chủ yếu dựa trên tri thức, prompt, bộ nhớ hoặc chiến lược xử lý được thiết kế sẵn. Điều này khiến chúng khó thích nghi lâu dài với môi trường mạng luôn thay đổi. Từ đó, nhu cầu xây dựng các **Self-Evolving AI Agents** trở nên cần thiết. Đây là các agent có khả năng tự cải thiện dựa trên dữ liệu mới, phản hồi từ người dùng, kết quả xử lý trong quá khứ và kinh nghiệm tích lũy trong quá trình tương tác.

Chính nhu cầu đó đã trở thành động lực chính cho nghiên cứu này. Trong dự án này, chúng tôi tập trung nghiên cứu và xây dựng một hệ thống AI Agent có khả năng tự cải thiện nhằm hỗ trợ phân tích và xử lý sự cố mạng viễn thông. Hệ thống hướng tới khả năng học liên tục từ dữ liệu vận hành, cập nhật bộ nhớ, tự phản tư sau mỗi lần xử lý và sử dụng các cơ chế phản hồi để cải thiện chất lượng chẩn đoán theo thời gian. Mục tiêu cuối cùng là hỗ trợ kỹ sư mạng trong việc phát hiện sự cố, phân tích nguyên nhân gốc rễ và đề xuất hướng xử lý phù hợp, đồng thời đảm bảo kết quả đưa ra có tính giải thích và có thể hỗ trợ quá trình ra quyết định trong thực tế.

1.2 Phát biểu bài toán

Mục tiêu của dự án là xây dựng một hệ thống **Self-Evolving AI Agents** hỗ trợ xử lý sự cố mạng viễn thông, trong đó agent có khả năng tiếp nhận dữ liệu vận hành mạng, phân tích các dấu hiệu bất thường, xác định nguyên nhân có khả năng cao nhất và đề xuất hướng xử lý phù hợp. Khác với các hệ thống chẩn đoán tĩnh, hệ thống được đề xuất cần có khả năng cải thiện theo thời gian thông qua việc học từ lịch sử xử lý, phản hồi của người dùng và kết quả đánh giá sau mỗi lần tương tác.

Cụ thể, bài toán được xem xét trong dự án này bao gồm ba nhóm nhiệm vụ chính. Thứ nhất, hệ thống cần phân tích dữ liệu đầu vào mô tả trạng thái mạng, bao gồm các chỉ số hiệu năng, thông tin cảnh báo, tham số cấu hình và các dữ liệu liên quan đến thiết bị hoặc topology mạng. Thứ hai, hệ thống cần thực hiện suy luận để xác định nguyên nhân gốc rễ của sự cố dựa trên các triệu chứng quan sát được. Thứ ba, hệ thống cần ghi nhận kết quả xử lý, phản hồi và các kinh nghiệm phát sinh để cập nhật bộ nhớ hoặc chiến lược suy luận, từ đó nâng cao hiệu quả xử lý trong các tình huống tương tự về sau.

Đầu vào và đầu ra của bài toán được định nghĩa như sau:

Đầu vào: Dữ liệu vận hành và giám sát mạng viễn thông, có thể bao gồm log hệ thống, cảnh báo mạng, chỉ số hiệu năng, thông tin cấu hình thiết bị, dữ liệu topology, lịch sử xử lý sự cố và phản hồi từ người dùng hoặc chuyên gia.

Đầu ra: Hệ thống AI Agent có khả năng hỗ trợ kỹ sư mạng thực hiện các tác vụ chính: phát hiện hoặc nhận diện sự cố, phân tích nguyên nhân gốc rễ, đưa ra giải thích cho quá trình suy luận, đề xuất hướng khắc phục và cập nhật kinh nghiệm xử lý nhằm cải thiện hiệu quả trong các lần chẩn đoán tiếp theo.

1.3 Đóng góp của dự án

Đóng góp chính của dự án nằm ở việc nghiên cứu và phát triển một hệ thống tự động chẩn đoán sự cố mạng truyền thông dựa trên hướng tiếp cận **Self-Evolving AI Agents** tích hợp trên nền tảng benchmark **NIKA**. Cụ thể hơn, các đóng góp chính của dự án bao gồm:

- **Xây dựng quy trình chẩn đoán sự cố trên nền tảng NIKA:** Nghiên cứu và ứng dụng môi trường giả lập mạng Kathará để triển khai các kịch bản sự cố phức tạp (như BGP ASN mismatch, link down, host crash, flow rule loop). Đồng thời thiết lập các AI Agent chẩn đoán hoạt động dựa trên các luồng suy luận phổ biến như ReAct, Plan & Execute, và Reflexion.
- **Đề xuất và hiện thực hóa mô-đun Procedural Memory:** Thiết kế hệ thống lưu trữ kỹ năng chẩn đoán dạng `ProceduralSkill` trong file JSON có cấu trúc, lấy cảm hứng từ hướng procedural memory của Skill-Pro. Mô-đun này cho phép agent ghi nhận các quy trình chẩn đoán có giá trị, truy xuất kỹ năng phù hợp với ngữ cảnh sự cố và tái sử dụng kinh nghiệm trong các incident tiếp theo.
- **Đề xuất và hiện thực hóa mô-đun Tool Refinement:** Thiết kế cơ chế cho phép AI Agent tự tinh chỉnh mô tả của các công cụ MCP nguyên thủy thông qua vòng lặp Explorer–Analyzer–Rewriter lấy cảm hứng từ DRAFT. Tài liệu tinh chỉnh được gắn động vào mô tả công cụ tại thời điểm thực thi, giúp LLM sử dụng công cụ chính xác và hiệu quả hơn theo thời gian. Cơ chế đóng băng adaptive đảm bảo dừng tinh chỉnh khi tài liệu hội tụ.
- **Đánh giá thực nghiệm chi tiết và sâu sắc:** Thực hiện hai lần chạy benchmark trên 125 case chung của NIKA để so sánh baseline và agent tự tiến hóa. Hệ thống

được đánh giá theo các chỉ số detection, localization, RCA, incident success rate, số bước chẩn đoán, số lần gọi công cụ và lượng token tiêu thụ; kết quả cho thấy incident success rate tăng từ 20.8% lên 28.0%, RCA F1 tăng từ 0.304 lên 0.387 và Localization F1 tăng từ 0.411 lên 0.459.

- **Công bố mã nguồn:** Mã nguồn hệ thống, cấu hình chạy và các thành phần thực nghiệm được đặt tại repository GitHub [thanhhai107/agent-net](https://github.com/thanhhai107/agent-net).
- **Phân tích thách thức trong thực tế:** Chỉ ra các thách thức còn tồn tại khi áp dụng Self-Evolving AI Agents vào môi trường mạng viễn thông thực tế, chẳng hạn như chất lượng dữ liệu vận hành, độ tin cậy của phản hồi, rủi ro học sai từ kinh nghiệm quá khứ, khả năng kiểm soát quá trình tự cập nhật và yêu cầu đảm bảo an toàn trong các hệ thống quan trọng.

Chương 2

Kiến thức nền tảng

Chương này tóm tắt các khái niệm nền tảng cần thiết cho đề tài: xử lý sự cố mạng, phân tích nguyên nhân gốc rễ, mô hình ngôn ngữ lớn, AI Agent và hướng tiếp cận Self-Evolving AI Agents.

2.1 Xử lý sự cố mạng và phân tích nguyên nhân gốc rễ

Mạng viễn thông hiện đại bao gồm nhiều lớp thiết bị, giao thức và dịch vụ phụ thuộc chặt chẽ với nhau. Một thay đổi nhỏ trong cấu hình định tuyến, trạng thái liên kết, dịch vụ nền hoặc chính sách điều khiển luồng có thể gây lỗi dây chuyền tới nhiều thành phần. Vì vậy, xử lý sự cố mạng không chỉ dừng ở phát hiện bất thường, mà còn cần định vị thiết bị lỗi và xác định nguyên nhân gốc rễ.

Trong đề tài này, quy trình chẩn đoán được chia thành ba tác vụ chính. Thứ nhất là **detection**, xác định mạng có đang gặp sự cố hay không. Thứ hai là **localization**, xác định thiết bị hoặc thành phần bị ảnh hưởng. Thứ ba là **root cause analysis** (RCA), xác định tên nguyên nhân gốc rễ. Về tổng quát, RCA có thể được xem là bài toán suy luận ngược từ trạng thái mạng, log, cấu hình và triệu chứng quan sát được để tìm nguyên nhân phù hợp nhất:

$$\hat{c} = \operatorname{argmax}_{c \in C} p(c \mid U, Y_t, s_t), \quad (2.1)$$

trong đó U là cấu hình mạng, Y_t là dữ liệu quan sát, s_t là triệu chứng tại thời điểm t và C là tập nguyên nhân có thể xảy ra.

2.2 Mô hình ngôn ngữ lớn và AI Agent

Mô hình ngôn ngữ lớn (LLM) có khả năng hiểu văn bản kỹ thuật, lập luận nhiều bước, sinh giải thích và sử dụng công cụ thông qua API. Trong bài toán xử lý sự cố mạng, LLM có thể đọc mô tả sự cố, phân tích log, gọi công cụ kiểm tra cấu hình và tổng hợp bằng chứng để đưa ra kết luận. Tuy nhiên, LLM thuần túy dễ gặp các hạn chế như thiếu tri thức miền cụ thể, suy luận thiếu kiểm chứng hoặc gọi công cụ sai tham số.

AI Agent mở rộng LLM bằng cách đặt mô hình vào một vòng lặp quan sát–suy luận–hành động. Agent có thể nhận nhiệm vụ, chọn công cụ phù hợp, thực thi truy vấn trên môi trường mạng, quan sát kết quả và tiếp tục điều tra cho đến khi đủ bằng chứng. Các workflow phổ biến gồm ReAct [8], Plan & Execute [6] và Reflexion [5]. Điểm mạnh của agent là kết hợp suy luận ngôn ngữ với dữ liệu thực tế từ công cụ, giúp kết quả chẩn đoán có cơ sở hơn so với câu trả lời sinh trực tiếp.

2.3 Self-Evolving AI Agents

Self-Evolving AI Agents là các agent có khả năng tự cải thiện từ kinh nghiệm trong quá trình hoạt động. Thay vì dùng một prompt, bộ nhớ hoặc mô tả công cụ cố định, agent có thể cập nhật tri thức sau mỗi ca xử lý dựa trên kết quả đánh giá, phản hồi hoặc vết tương tác. Điều này phù hợp với môi trường mạng biến thông vì topology, giao thức, dịch vụ và dạng lỗi có thể thay đổi theo thời gian.

Các thành phần thường có thể tiến hóa gồm:

- **Bộ nhớ:** Lưu các quy trình chẩn đoán, bài học hoặc mẫu lỗi đã được kiểm chứng để tái sử dụng trong các incident tương tự.
- **Ngữ cảnh và prompt:** Cải thiện cách đưa thông tin vào mô hình, giúp agent tập trung vào bằng chứng quan trọng.
- **Công cụ:** Cải thiện mô tả, ví dụ sử dụng và điều kiện tiên đề của công cụ để giảm lỗi gọi công cụ.
- **Chiến lược suy luận:** Điều chỉnh thứ tự kiểm tra giả thuyết, cách loại trừ nguyên nhân và cách dừng điều tra.

Trong nghiên cứu này, hệ thống tập trung vào hai hướng tiến hóa có rủi ro triển khai thấp hơn so với fine-tuning mô hình: tiến hóa bộ nhớ quy trình và tiến hóa tài liệu công cụ.

2.4 Các nghiên cứu liên quan trực tiếp

Đề tài chỉ tập trung vào hai hướng nghiên cứu liên quan trực tiếp đến phần hiện thực. Thứ nhất, Skill-Pro nghiên cứu procedural memory cho LLM agent, trong đó kinh nghiệm xử lý được biểu diễn thành các kỹ năng có thể truy xuất và tái sử dụng trong quá trình suy luận dài [3]. Ý tưởng này phù hợp với bài toán xử lý sự cố mạng vì mỗi incident thành công có thể trở thành một quy trình chẩn đoán hữu ích cho các tình huống tương tự.

Thứ hai, DRAFT tập trung vào việc cải thiện khả năng sử dụng công cụ của LLM thông qua tinh chỉnh tài liệu mô tả công cụ [7]. Thay vì thay đổi mã nguồn công cụ, hệ thống học từ các lần gọi công cụ thực tế để bổ sung mô tả, điều kiện sử dụng, ràng buộc tham số và hướng dẫn diễn giải kết quả trả về.

Từ hai hướng này, đề tài xây dựng hai mô-đun tương ứng: **Procedural Memory** cho tiến hóa bộ nhớ quy trình và **Tool Refinement** cho tinh chỉnh tài liệu công cụ. Cách tiếp cận này giúp agent cải thiện hành vi mà không cần cập nhật tham số mô hình nền, nhờ đó dễ kiểm soát và phù hợp hơn với yêu cầu an toàn của hệ thống vận hành mạng.

Chương 3

Kiến trúc hệ thống

Chương này trình bày kiến trúc hệ thống Self-Evolving AI Agents được xây dựng trên nền tảng NIKA. Mục tiêu của hệ thống là cho phép agent tương tác với môi trường mạng giả lập, thu thập bằng chứng chẩn đoán, đưa ra kết luận detection/localization/RCA và cải thiện hiệu quả qua các incident liên tiếp.

3.1 Kiến trúc tổng quan

Hệ thống gồm bốn thành phần chính:

1. **Môi trường mạng NIKA/Kathará:** Khởi tạo các topology mạng ảo hóa bằng Docker container, hỗ trợ nhiều giao thức và công nghệ như BGP, OSPF, RIP, SDN, P4/BMv2, DNS, DHCP và HTTP.
2. **AI Agent chẩn đoán:** Sử dụng LLM làm lõi suy luận, vận hành theo workflow ReAct, Plan & Execute hoặc Reflexion trên LangGraph [2].
3. **MCP Servers:** Cung cấp giao diện chuẩn hóa để agent gọi công cụ kiểm tra kết nối, cấu hình host, bảng định tuyến, trạng thái dịch vụ, log P4/BMv2, telemetry và nộp kết quả chẩn đoán [1].
4. **Mô-đun tự tiến hóa:** Bao gồm Procedural Memory để lưu và truy xuất kỹ năng chẩn đoán, cùng Tool Refinement để tinh chỉnh tài liệu công cụ.

Trong mỗi incident, NIKA khởi động lab tương ứng, tiêm lỗi vào môi trường mạng, sau đó agent thực hiện chẩn đoán bằng cách gọi các công cụ MCP. Kết quả cuối cùng được gửi qua Task Management MCP Server dưới dạng `is_anomaly`, `faulty_devices`

và `root_cause_name`. Sau khi episode kết thúc, hệ thống đánh giá kết quả và kích hoạt các mô-đun học ngoại tuyến.

3.2 Workflow chẩn đoán

Workflow chính được sử dụng trong thực nghiệm là ReAct. Agent luân phiên giữa bước suy luận và bước gọi công cụ: từ triệu chứng ban đầu, agent đặt giả thuyết, chọn công cụ kiểm chứng, đọc kết quả và cập nhật hướng điều tra. Giới hạn `max_steps` giúp tránh vòng lặp vô hạn và kiểm soát chi phí.

Hệ thống cũng hỗ trợ Plan & Execute và Reflexion. Plan & Execute tách quá trình lập kế hoạch khỏi thực thi, phù hợp với các sự cố cần chuỗi điều tra dài. Reflexion bổ sung bước tự đánh giá và thử lại khi kết quả chưa đạt yêu cầu. Trong phạm vi đánh giá chính, ReAct được chọn làm workflow so sánh vì đơn giản, ổn định và là baseline phổ biến cho agent sử dụng công cụ.

3.3 Procedural Memory

Procedural Memory lưu trữ kinh nghiệm chẩn đoán dưới dạng `ProceduralSkill`. Mỗi kỹ năng gồm điều kiện kích hoạt, chuỗi bước thực hiện, điều kiện kết thúc và các thuộc tính ngữ cảnh như `scenario`, `protocol`, `service`, `symptom` và `tool`. Trạng thái bộ nhớ được lưu trong `runtime/memory/{bank_id}/skills.json`. Thiết kế này lấy cảm hứng từ hướng procedural memory của Skill-Pro.

Trước mỗi episode, hệ thống sinh `MemoryQuery` từ ngữ cảnh public của incident và truy xuất các kỹ năng phù hợp. Điểm truy xuất kết hợp độ khớp từ khóa, độ khớp `scenario`, độ khớp thuộc tính và độ tin cậy của kỹ năng. Trong quá trình agent gọi công cụ, `SkillToolRuntime` có thể chèn hướng dẫn ngắn từ kỹ năng đang hoạt động để agent chọn đúng công cụ, đúng tham số và biết khi nào cần dừng hoặc chuyển giả thuyết.

Sau episode, hệ thống tính phần thưởng dựa trên detection, localization, RCA và chi phí thao tác. Các episode tốt được đưa vào `experience pool`. Từ đó, LLM sinh semantic gradient để đề xuất cải thiện kỹ năng; ứng viên kỹ năng mới chỉ được chấp nhận khi vượt qua PPO-style gate [4]. Cơ chế này giúp bộ nhớ học từ ca thành công nhưng hạn chế việc ghi nhận kinh nghiệm nhiều hoặc quá khớp.

3.4 Tool Refinement

Tool Refinement cải thiện cách agent hiểu và sử dụng công cụ MCP bằng cách tinh chỉnh tài liệu mô tả công cụ, không thay đổi mã nguồn thực thi. Trạng thái được lưu tại `runtime/tool_evolution/{library_id}/state.json`. Thiết kế này lấy cảm hứng từ cơ chế tinh chỉnh tài liệu công cụ của DRAFT.

Sau mỗi session, hệ thống trích xuất các lần gọi công cụ từ `messages.jsonl`, xác định các khoảng trống hiểu biết như thiếu mô tả tham số, sai tên thiết bị, thiếu điều kiện tiên đề hoặc chưa biết cách diễn giải output. Analyzer sinh gợi ý cải thiện, Rewriter viết lại mô tả công cụ với ràng buộc tham số, ví dụ đúng/sai và ghi chú diễn giải kết quả. Khi tài liệu đạt ngưỡng hội tụ, công cụ được đóng băng để tránh viết lại không cần thiết.

Hai mô-đun Procedural Memory và Tool Refinement hỗ trợ cho nhau: Procedural Memory cải thiện chiến lược chẩn đoán ở mức quy trình, còn Tool Refinement cải thiện độ chính xác khi sử dụng từng công cụ cụ thể.

Chương 4

Thực nghiệm

Chương này trình bày thiết lập thực nghiệm và kết quả đánh giá hệ thống Self-Evolving AI Agents trên benchmark NIKA. Mục tiêu chính là so sánh agent ReAct thông thường với agent ReAct được bổ sung Procedural Memory và Tool Refinement.

4.1 Thiết lập thực nghiệm

Thực nghiệm so sánh hai cấu hình agent:

- **Baseline Agent:** ReAct agent tiêu chuẩn, không sử dụng bộ nhớ tiến hóa và không tinh chỉnh tài liệu công cụ.
- **Evolved Agent:** ReAct agent tích hợp Procedural Memory và Tool Refinement, cho phép tái sử dụng kỹ năng chẩn đoán và tài liệu công cụ đã được cải thiện từ các episode trước.

Cả hai cấu hình sử dụng cùng mô hình nền `openai/gpt-oss-120b`, cùng tập MCP servers và giới hạn `max_steps=20`. Với Baseline Agent, mỗi incident được xử lý độc lập. Với Evolved Agent, kinh nghiệm từ các incident trước được cập nhật ngoại tuyến và sử dụng cho các incident sau.

Quy trình chạy benchmark gồm bốn bước: khởi động lab Kathará, tiêm lỗi xác định, chạy agent chẩn đoán và đánh giá kết quả. Môi trường thực nghiệm sử dụng Python 3.12, LangGraph/LangChain cho orchestration, FastMCP cho MCP servers và Kathará cho ảo hóa mạng.

4.2 Benchmark và chỉ số đánh giá

Hai lần chạy benchmark gồm 125 case chung thuộc 6 nhóm nguyên nhân gốc rễ trên 8 kịch bản mạng: OSPF enterprise, BGP CLOS data center, SDN CLOS, RIP/VPN và P4/BMv2. Các nhóm lỗi bao gồm link failures, end-host failures, misconfigurations, resource contention, network node errors và network under attack; ngoài ra có các case `no_fault` để kiểm tra khả năng không báo động sai.

Mỗi incident có ground truth gồm ba trường: trạng thái bất thường `is_anomaly`, danh sách thiết bị lỗi `faulty_devices` và tên nguyên nhân gốc rễ `root_cause_name`. Bảng kết quả tổng thể được tính trực tiếp từ hai batch `benchmark_evaluate-0001` và `benchmark_evaluate-0002`.

Hệ thống được đánh giá theo ba nhóm chỉ số:

- **Detection:** Agent xác định đúng mạng có sự cố hay không.
- **Localization:** Agent xác định đúng thiết bị lỗi, đo bằng accuracy và F1 trên tập thiết bị.
- **RCA:** Agent xác định đúng nguyên nhân gốc rễ, đo bằng accuracy và F1.

Một incident được xem là thành công toàn diện khi agent đồng thời phát hiện đúng sự cố, định vị đúng toàn bộ thiết bị lỗi và xác định đúng nguyên nhân gốc rễ. Ngoài độ chính xác, thực nghiệm cũng đo số bước suy luận, số lần gọi công cụ và lượng token trung bình.

4.3 Kết quả thực nghiệm

Bảng 4.1 trình bày kết quả chung của cấu hình ReAct + GPT-OSS-120B, so sánh Baseline Agent và Evolved Agent.

Kết quả cho thấy Evolved Agent cải thiện trên hầu hết các chỉ số chẩn đoán. Incident success rate tăng từ 20.8% lên 28.0% (26/125 lên 35/125), tương đương tăng 34.6%. RCA F1 tăng từ 0.304 lên 0.387, Localization F1 tăng từ 0.411 lên 0.459. Đồng thời, agent tự tiến hóa dùng ít bước hơn (giảm 16.7%), gọi công cụ ít hơn (giảm 18.5%) và giảm tool error rate từ 0.65% xuống 0.00%.

Mức cải thiện rõ nhất xuất hiện ở các lỗi cấu hình host như thiếu IP, sai IP hoặc sai gateway. Với các lỗi này, Procedural Memory giúp agent tái sử dụng quy trình kiểm tra cấu hình host, còn Tool Refinement giúp mô tả công cụ rõ hơn về tham số và cách diễn

Bảng 4.1: Kết quả chung của ReAct + GPT-OSS-120B

Chỉ số	Baseline	Evolved
Detection Score	0.840	0.872
Localization Accuracy	0.368	0.384
Localization F1	0.411	0.459
RCA Accuracy	0.304	0.376
RCA F1	0.304	0.387
Incident Success Rate	26/125 (20.8%)	35/125 (28.0%)
Steps trung bình	15.29	12.74
Tool calls trung bình	13.60	11.09
Tool error rate	11/1700 (0.65%)	0/1386 (0.00%)
Input tokens (tổng)	15.944M	17.992M
Output tokens (tổng)	0.697M	0.708M
Total tokens (tổng)	16.641M	18.699M
Tổng thời gian chạy	207.7 phút	222.6 phút

giải output. Ở các lỗi BGP/OSPF phức tạp, SDN/P4 hoặc resource contention, cải thiện vẫn còn hạn chế vì cần suy luận nhiều tầng và phụ thuộc mạnh vào bằng chứng từ nhiều công cụ.

4.3.1 Các bảng và hình bổ sung dự kiến

Các bảng và hình trong phần này là placeholder để cố định bố cục báo cáo. Các giá trị được đánh dấu **minh họa** và sẽ được thay bằng kết quả thực nghiệm chính thức sau khi hoàn tất các batch benchmark tương ứng.

Bảng 4.2: Placeholder table: so sánh ReAct, Plan&Execute và Reflexion trên GPT-OSS-120B. Số liệu minh họa, chưa dùng làm kết quả chính thức.

Chiến lược agent	Incident Success	RCA F1	Tool calls
ReAct	28.0%	0.387	11.09
Plan&Execute	25.6%	0.361	12.74
Reflexion	26.4%	0.374	13.28

Minh họa xu hướng metric theo số case tăng dần				
Case	RCA F1		Localization F1	
	Baseline	Evolved	Baseline	Evolved
25	0.22	0.43	0.34	0.55
50	0.28	0.42	0.39	0.53
75	0.30	0.40	0.40	0.49
100	0.31	0.39	0.41	0.47
125	0.30	0.39	0.41	0.46

Đường cong dự kiến thể hiện cải thiện nhanh ở nhóm case đầu do có nhiều case liên quan, sau đó chậm lại khi gặp nhóm case xa hơn.

Hình 4.1: Placeholder figure: xu hướng RCA và Localization theo số case tăng dần. Số liệu minh họa, chưa dùng làm kết quả chính thức.

Minh họa độ ổn định qua 3 lần chạy ReAct + GPT-OSS-120B				
Run	Incident Success		RCA F1	
	Baseline	Evolved	Baseline	Evolved
1	20.8%	28.0%	0.304	0.387
2	19.6%	27.2%	0.296	0.381
3	21.6%	29.6%	0.311	0.395

Figure chính thức sẽ dùng error bar hoặc box plot để thể hiện trung bình và độ lệch chuẩn giữa các lần chạy.

Hình 4.2: Placeholder figure: so sánh độ ổn định giữa baseline và evolved qua 3 lần chạy. Số liệu minh họa, chưa dùng làm kết quả chính thức.

Minh họa so sánh mô hình nền với ReAct				
Model	Baseline		Evolved	
	Success	RCA F1	Success	RCA F1
GPT-OSS-120B	20.8%	0.304	28.0%	0.387
Qwen3.5-122B-A10B-FP8	18.4%	0.286	25.6%	0.365

Figure chính thức sẽ dùng grouped bar chart để so sánh ReAct trên hai model nền.

Hình 4.3: Placeholder figure: so sánh ReAct trên GPT-OSS-120B và Qwen3.5-122B-A10B-FP8. Số liệu minh họa, chưa dùng làm kết quả chính thức.

4.4 Phân tích hiệu quả và hạn chế

Kết quả thực nghiệm xác nhận rằng hướng tiếp cận tự tiến hóa có thể cải thiện hiệu quả chẩn đoán mà không cần fine-tuning mô hình nền. Procedural Memory đóng vai trò tích lũy quy trình chẩn đoán ở mức chiến lược, giúp agent chọn thứ tự kiểm tra hợp lý hơn. Tool Refinement cải thiện hiểu biết ở mức công cụ, giảm lỗi gọi tham số và giúp agent diễn giải output chính xác hơn.

Đổi lại, Evolved Agent tiêu thụ nhiều token hơn Baseline Agent: tổng token tăng từ 16.641M lên 18.699M (+12.4%), trong đó input tokens tăng từ 15.944M lên 17.992M và output tokens tăng nhẹ từ 0.697M lên 0.708M. Tổng thời gian chạy tăng từ 207.7 lên 222.6 phút (+7.2%). Đây là chi phí chấp nhận được trong bối cảnh incident success rate tăng 34.6%, nhưng vẫn cần tối ưu nếu triển khai trong môi trường vận hành quy mô lớn.

Một hạn chế khác là khả năng tổng quát hóa chưa đồng đều giữa các nhóm sự cố. Các kỹ năng học từ incident trước có thể phù hợp tốt với lỗi host hoặc lỗi cấu hình đơn giản, nhưng chưa đủ mạnh cho các sự cố liên quan đến định tuyến động, tấn công mạng hoặc tương tác phức tạp giữa nhiều thành phần. Điều này cho thấy cần tiếp tục cải thiện cơ chế trừu tượng hóa kỹ năng và chọn lọc kinh nghiệm trong các phiên bản tiếp theo.

Chương 5

Kết luận và hướng phát triển

Chương này tổng kết lại những kết quả đạt được của dự án nghiên cứu và phát triển hệ thống Self-Evolving AI Agents trên nền tảng NIKAI, đồng thời thảo luận thẳng thắn về những hạn chế hiện tại và đề xuất định hướng phát triển trong tương lai.

5.1 Kết quả đạt được của dự án

Sau quá trình nghiên cứu và thực nghiệm nghiêm túc, dự án đã đạt được một số kết quả quan trọng sau:

- **Nghiên cứu lý thuyết có trọng tâm:** Tổng hợp và hệ thống hóa hai hướng nghiên cứu liên quan trực tiếp đến đề tài: procedural memory trong Skill-Pro và tinh chỉnh tài liệu công cụ trong DRAFT.
- **Xây dựng kiến trúc hệ thống hoàn chỉnh:** Tích hợp thành công AI Agent với môi trường giả lập mạng Kathará thông qua giao thức Model Context Protocol (MCP) chuẩn hóa. Hệ thống vận hành tốt trên 3 luồng chẩn đoán phổ biến (ReAct, Plan & Execute, Reflexion).
- **Hiện thực hóa thành công các mô-đun tiến hóa:**
 1. Mô-đun Procedural Memory lưu trữ `ProceduralSkill` trong file JSON (`runtime/memory/{bank_id}/skills.json`), hỗ trợ truy xuất kỹ năng theo ngữ cảnh và tiến hóa kỹ năng ngoại tuyến.
 2. Mô-đun Tool Refinement tự tinh chỉnh mô tả các công cụ MCP nguyên thủy thông qua vòng lặp Explorer–Analyzer–Rewriter. Trạng thái được lưu tại thư

mục `runtime/tool_evolution/` với cơ chế đóng băng adaptive khi hội tụ.

- **Đánh giá thực nghiệm chi tiết:** Kiểm chứng hệ thống qua hai lần chạy benchmark trên 125 case chung của NIKA. Kết quả cho thấy Evolved Agent đạt incident success rate 28.0% (35/125), tăng 34.6% so với Baseline Agent 20.8% (26/125). RCA F1 cải thiện từ 0.304 lên 0.387, số bước chẩn đoán giảm 16.7%, số lần gọi công cụ giảm 18.5% và tool error rate giảm từ 0.65% xuống 0.00%.
- **Repository dự án:** Mã nguồn và cấu hình thực nghiệm được đặt tại `thanhhai107/agent-net`.

5.2 Những hạn chế hiện tại

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế cần khắc phục:

- **Khả năng tổng quát hóa giữa các kịch bản còn hạn chế:** Các kỹ năng và tài liệu công cụ được tiến hóa từ những episode trước có thể thiên về cấu trúc topology, tên thiết bị hoặc giao thức đã gặp. Vì vậy, khi chuyển sang các kịch bản phức tạp hơn như BGP/OSPF, SDN, P4 hoặc các lỗi resource contention, agent vẫn có thể chọn sai hướng điều tra, gọi công cụ với tham số chưa phù hợp hoặc chưa suy luận đủ sâu từ bằng chứng thu thập được.
- **Tăng chi phí Token và thời gian chạy:** Việc tải thêm tài liệu công cụ đã được tinh chỉnh và các kỹ năng từ Procedural Memory vào ngữ cảnh prompt làm tổng token tăng 12.4% (từ 16.641M lên 18.699M token), chủ yếu đến từ input tokens. Tổng thời gian chạy tăng 7.2% (từ 207.7 lên 222.6 phút).
- **Chất lượng tiến hóa phụ thuộc vào LLM:** Cả Semantic Gradient trong Procedural Memory lẫn Rewriter trong Tool Refinement đều dựa vào LLM để sinh đề xuất cải tiến. Khi LLM sinh gradient hoặc tài liệu sai lệch, hệ thống phải dùng fallback xác định, làm giảm chất lượng học hỏi.

5.3 Định hướng phát triển tương lai

Từ những kết quả và hạn chế trên, dự án định hướng một số nội dung nghiên cứu tiếp theo bao gồm:

- **Cải thiện khả năng tổng quát hóa (Generalization):** Nghiên cứu các cơ chế trừu tượng hóa ngữ nghĩa mạnh mẽ hơn trong Procedural Memory và Tool Refinement, giúp kỹ năng chẩn đoán và tài liệu công cụ tinh chỉnh dễ dàng thích ứng với topology, tên thiết bị, giao thức và nhóm lỗi mới chưa gặp.
- **Tích hợp cơ chế Human-in-the-loop:** Bổ sung giao diện cho phép kỹ sư mạng tham gia phản hồi, đánh giá và phê duyệt các kỹ năng mới được PPO Gate chấp nhận cũng như tài liệu công cụ đã tinh chỉnh trước khi ghi nhận chính thức vào bộ nhớ.
- **Tối ưu hóa hiệu năng và chi phí:** Tinh chỉnh các mô hình ngôn ngữ lớn chuyên biệt cho miền mạng (Domain-specific LLMs) để giảm kích thước prompt chẩn đoán, tối ưu hóa thời gian thực thi các Kathará labs, và giảm chi phí token khi tải kỹ năng từ Procedural Memory.
- **Mở rộng kịch bản sự cố:** Đánh giá agent tự tiến hóa trên các kịch bản mạng có quy mô lớn hơn (topology tier m, l) và các sự cố phức tạp hơn (multi-fault incidents với nhiều nguyên nhân đồng thời).

Tài liệu tham khảo

- [1] Anthropic, “Model context protocol,” <https://modelcontextprotocol.io>, 2024.
- [2] H. Chase *et al.*, “Langgraph: Building stateful, multi-actor applications with llms,” *LangChain Blog*, 2024.
- [3] C. Qian *et al.*, “Scaling llm test-time compute for long-context reasoning via procedural memory,” *arXiv preprint*, 2024.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [6] G. Wang *et al.*, “Plan-and-execute: Planning with language models for agentic tasks,” *arXiv preprint*, 2023.
- [7] J. Xu *et al.*, “Draft: Difficulty-responsive adaptive fine-tuning for tool use in large language models,” *arXiv preprint*, 2024.
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.